

A1: RESEARCH QUESTION

Can we perform sentiment analysis on the text reviews for the combined data from Amazon, IMDb, and Yelp using a neural network model and NLP techniques, in order to understand areas for improvement based on our customers' feedback and enhance customer satisfaction?

A2: OBJECTIVES OR GOAL

According to Virahonda, Sergio (2020, October 9), sentiment analysis is a term used in natural language processing and Machine learning to interpret and classify emotions in text information. According to Geek for geeks (2025, July 12), sentiment analysis is the process of classifying whether a block of text is positive, negative, or neutral, which helps businesses use customers' opinions to improve their services and products.

The goal of this data analysis is to develop a neural network model that is capable of performing text classification on the combined dataset of Amazon, IMDb, and Yelp, and use it to predict text selections with the aim of performing sentiment analysis so as to understand the customer satisfaction trends on the various products and services offered and identify the areas of improvement.

B1: DATA EXPLORATION

- presence of unusual characters (e.g., emojis, non-English characters)

There were no emojis found in the data, whereas 15 unusual characters were identified, as shown in the screenshot below.

```
#source of regex : https://pythonguides.com/check-if-a-string-is-an-emoji-in-python/

#Checking for unusual characters (e.g., emojis, non-English characters)
#Removing nulls from the data frame and
texts = Sentiment_df["text"].dropna().astype(str).tolist()

# Regex for emojis
emoji_pattern = re.compile("[
    u\"\\U0001F600-\\U0001F64F\" # identify emojis/emoticons
    u\"\\U0001F300-\\U0001F5FF\" # symbols & pictographs
    u\"\\U0001F680-\\U0001F6FF\" # transport & map symbols
    u\"\\U0001F700-\\U0001F77F\" # alchemical symbols
    u\"\\U0001F780-\\U0001F7FF\" # Geometric Shapes Extended
    u\"\\U0001F800-\\U0001F8FF\" # Supplemental Arrows-C
    u\"\\U0001F900-\\U0001F9FF\" # Supplemental Symbols and Pictographs
    u\"\\U0001FA00-\\U0001FA6F\" # Chess Symbols
    u\"\\U0001FA70-\\U0001FAFF\" # Symbols and Pictographs Extended-A
    u\"\\U0001F1E0-\\U0001F1FF\" # flags
"]+", flags=re.UNICODE)
# Count emojis
emoji_count = sum(len(emoji_pattern.findall(t)) for t in texts)

# Count non-ASCII characters (anything beyond standard English letters & punctuation)
non_ascii_chars = Counter(ch for t in texts for ch in t if ord(ch) > 127)

print("Total emoji occurrences:", emoji_count)
print("Top 15 unusual characters:", non_ascii_chars.most_common(20))
```

```
Total emoji occurrences: 0
Top 15 unusual characters: [('é', 7), ('\x96', 5), ('\x85', 2), ('â', 1), ('\x97', 1), ('ê', 1)]
```

- vocabulary size

Vocabulary size refers to the total number of unique words in a data set. In the case of the combined datasets of Amazon, Yelp, and IMDb, there were 5113 unique words

```
#Finding the vocabulary
Sentiment_reviews = Sentiment_df['text'].tolist()
vocabulary = set()
max_review_len = 0

for review in Sentiment_reviews:
    words = review.split()
    max_review_len = max(max_review_len, len(words))
    vocabulary.update(words)
vocab_size = len(vocabulary)

print("Vocabulary Size: ", vocab_size)
print("Longest Review Length: ", max_review_len)
```

```
Vocabulary Size: 5113
Longest Review Length: 785
```

- proposed word embedding length

Word embedding length is the number of values that is used to represent each word in a dense vector and is also referred to as the embedding dimension. Word embedding length is used to transform words into numeric vectors that capture their semantic meaning and contextual relationships, enabling machine learning models to understand them. For the combined dataset, the embedding length is 123.

```
#Finding the word embedding Length|

embedding_size = int(np.log2(vocab_size) * 10)
print("Embedding Size: ", embedding_size)

Embedding Size: 123
```

- statistical justification for the chosen maximum sequence length

Sequence length is the number of tokens in a text. In the case of the semantic analysis model, a sequence length of 21 was utilized as the maximum sequence length because it represents the 99-percentile quartile and hence allows at least there will be no truncation of data for 99% of the text reviews

```
#

#Padding process
#Padding is the process of making all sequences the same length
# Set max_len to the 95th percentile
Sentiment_df["sequence_length"] = Sentiment_df["text_sequence"].apply(len)
# Obtaining the quantiles Length
quantiles = Sentiment_df["sequence_length"].quantile([0.0, 0.1, 0.25, 0.5, 0.75, 0.9, 0.95, 0.99, 1.0])
print("Sequence Length Quantiles:\n", quantiles)
#Defining the 95th quantile to be used in the padding process
max_len = int(Sentiment_df["sequence_length"].quantile(0.99))
print("95th percentile max_len:", max_len)

Sentiment_padding = pad_sequences(Sentiment_df["text_sequence"], maxlen=max_len, padding='post')
Sentiment_df["text_padded"] = list(Sentiment_padding)

print(Sentiment_df.head(1))

Sequence Length Quantiles:
0.00    0.00
0.10    2.00
0.25    3.00
0.50    5.00
0.75    8.00
0.90   11.30
0.95   14.00
0.99   21.53
1.00   793.00
Name: sequence_length, dtype: float64
95th percentile max_len: 21
```

B2: TOKENIZATION

Tokenization is the process of splitting text into tokens or smaller, discrete units, enabling neural network models to analyze the text. Each token represents a meaningful unit of language.

According to Zakaria, Ahmed, tokenization serves as a transitional bridge between the richness of human language and the structured, numerical requirements of computational models.

In the process of tokenizing the words, I followed the process below

- I started by applying the word tokenization to break the text into individual words. Word tokenization is the process of splitting text into tokens or smaller, discrete units so as to enable neural network models to analyze the text
- I then matched the word using the lemmatization process. Lemmatization uses a linguistic dictionary, rules, and part-of-speech tags to return the actual valid word
- I then converted the words to tokens and created the text sequence

The following libraries were used for the tokenization process

- For breaking the word into individual texts, I used word_tokenize from nltk.tokenize package
- For matching the words, I used WordNetLemmatizer from nltk.stem package
- For tokenization, I used the Tokenizer from the package tensorflow.keras.preprocessing

The goal of the tokenization process is to

- Convert the text into neural network model-readable inputs, as the neural network works with numbers.
- Standardize the input granularity by defining the level of granularity of the text.
- Reduce the size of the vocabulary
- Preserve the semantic meaning of the original text

- Prepare the data for the consistent sequence lengths.

The code

```
# Word Tokenization
#Word Tokenization is the process of splitting text into smaller pieces or tokens.This enables NLP to be able to analyse text
# Word Tokenization of the data

Sentiment_df["text_tokenized"] = Sentiment_df["text"].apply(word_tokenize)

#Stemming and Lemmatizing(Matching words)
#Stemming is a text pre-processing technique in NLP that reduces words to their root formby removing prefixes and suffixes
#Lematization uses linguistics dictionary, rules amd part of speech tags to return avtual valid word
#Steming is faster , rule based but it cab produce weierd non-words
#Lemati#zation is slower but gives better words for readability and analysis

#Lemmatization of the data

lemmatizer = WordNetLemmatizer()
Sentiment_df["text_lemmatized"] = Sentiment_df["text_tokenized"].apply(
    lambda x: " ".join([lemmatizer.lemmatize(word) for word in x]))

pd.set_option("display.max_colwidth", None)
Sentiment_df.head()
```

	text	Rating	total Words	text_tokenized	text_lemmatized
0	way plug us unless go converter	0	21	[way, plug, us, unless, go, converter]	way plug u unless go converter
1	good case excellent value	1	4	[good, case, excellent, value]	good case excellent value
2	great jawbone	1	4	[great, jawbone]	great jawbone
3	tied charger conversations lasting 45 minutes major problems	0	11	[tied, charger, conversations, lasting, 45, minutes, major, problems]	tied charger conversation lasting 45 minute major problem
4	mic great	1	4	[mic, great]	mic great

```
#Tokenize the text into individual words
tokenizer =Tokenizer()
tokenizer.fit_on_texts(Sentiment_df["text_lemmatized"])
Sentiment_df["text_sequence"] = tokenizer.texts_to_sequences(Sentiment_df["text_lemmatized"])
Sentiment_df.head()
```

	text	Rating	total Words	text_tokenized	text_lemmatized	text_sequence
0	way plug us unless go converter	0	21	[way, plug, us, unless, go, converter]	way plug u unless go converter	[44, 252, 103, 476, 25, 2043]
1	good case excellent value	1	4	[good, case, excellent, value]	good case excellent value	[1, 60, 34, 315]
2	great jawbone	1	4	[great, jawbone]	great jawbone	[3, 963]
3	tied charger conversations lasting 45 minutes major problems	0	11	[tied, charger, conversations, lasting, 45, minutes, major, problems]	tied charger conversation lasting 45 minute major problem	[2044, 126, 632, 964, 1326, 61, 1327, 77]
4	mic great	1	4	[mic, great]	mic great	[759, 3]

B3:PADDING PROCESS

- if the padding occurs before or after the text sequence

Text sequence is an ordered list of tokens. The tokens can be words, sub-words, and characters that represent a piece of the original text after tokenization. The padding process is the process of ensuring all sequences are the same length. The padding process was conducted after the text sequence had been processed. The padding process utilized the sequenced data and employed post-padding, where zeros were added after the text to prevent truncation. The Padding length is used for the maximum length of the 99th quantile, which is 21, and allows for the preservation of most of the text. The shorter sequence was padded with 0, whereas the longer sequence was truncated to fit the maximum sequence of 21.

- a screenshot of a single padded sequence

```
#Padding process
#Padding is the process of making all sequences the same length
# Set max_len to the 95th percentile
Sentiment_df["sequence_length"] = Sentiment_df["text_sequence"].apply(len)
# Obtaining the quantiles length
quantiles = Sentiment_df["sequence_length"].quantile([0.0, 0.1, 0.25, 0.5, 0.75, 0.9, 0.95, 0.99, 1.0])
print("Sequence Length Quantiles:\n", quantiles)
#Defining the 95th quantile to be used in the padding process
max_len = int(Sentiment_df["sequence_length"].quantile(0.99))
print("95th percentile max_len:", max_len)

Sentiment_padding = pad_sequences(Sentiment_df["text_sequence"], maxlen=max_len, padding='post')
Sentiment_df["text_padded"] = list(Sentiment_padding)

print(Sentiment_df.head(1))
```

```
Sequence Length Quantiles:
0.00    0.00
0.10    2.00
0.25    3.00
0.50    5.00
0.75    8.00
0.90   11.30
0.95   14.00
0.99   21.53
1.00   793.00
Name: sequence_length, dtype: float64
95th percentile max_len: 21
      text  Rating  total Words  \
0  way plug us unless go converter      0      21

      text_tokenized      text_lemmatized  \
0  [way, plug, us, unless, go, converter]  way plug u unless go converter

      text_sequence  sequence_length  \
0  [44, 252, 103, 476, 25, 2043]      6

      text_padded
0  [44, 252, 103, 476, 25, 2043, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
print(Sentiment_padding.shape)
```

```
(2748, 21)
```

B4: CATEGORIES OF SENTIMENT

The categories used for the activation function are two score ratings, 0 and 1, where 0 represents the negative and 1 represents the positive.

```
# Viewing the sentiments
Sentiment_df['Rating'].value_counts()
# Score is either 1 (for positive) or 0 (for negative)

|: Rating
1    1386
0    1362
Name: count, dtype: int64
```

B5: STEPS TO PREPARE THE DATA

In preparation of the data set, the following steps were taken

- Loading the libraries needed

The following libraries were loaded

```
# Importing the Libraries/ packages
!pip install "numpy<1.27" --upgrade
!pip install tensorflow
!pip install wordcloud
import pandas as pd
from pandas import Series, DataFrame
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder
import tensorflow as tf
import tensorflow.keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Flatten, GlobalAveragePooling1D, Bidirectional
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping

import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
nltk.download("stopwords")
from nltk.stem import WordNetLemmatizer
nltk.download("wordnet")
nltk.download("punkt")
from wordcloud import WordCloud, STOPWORDS, ImageColorGenerator
from PIL import Image
import re
from collections import Counter

import warnings
with warnings.catch_warnings():
    warnings.simplefilter("ignore", FutureWarning)
    warnings.filterwarnings("ignore")
import os
import datetime
import random
```

- Loading the data

Pandas read_csv was utilized to load the data

```
##Loading the data
#defining the file path
Local_path = 'C:/Users/beatr/OneDrive/Desktop/WGU/D213/Task2/Data/sentiment labelled sentences/'
#Input data
InputFilePath_imdb = Local_path + 'imdb_labelled.txt'
InputFilePath_amazon = Local_path + 'amazon_cells_labelled.txt'
InputFilePath_yelp = Local_path + 'yelp_labelled.txt'
#Output data
CleanedFilePath= Local_path + 'Sentiment_cleaned_data.txt'
X_TrainedDataFilePath= Local_path + 'X_Train_data.txt'
X_TestDataFilePath= Local_path + 'X_Test_data.txt'
y_TrainedDataFilePath= Local_path + 'Y_Train_data.txt'
y_TestDataFilePath= Local_path + 'y_Test_data.txt'

# Loading the data files
amazon_df = pd.read_csv(InputFilePath_amazon, sep="\t", header=None, names=["text", "Rating"])
print(amazon_df.shape)
#print(amazon_df.head())
imdb_df = pd.read_csv(InputFilePath_imdb, sep="\t", header=None, names=["text", "Rating"])
print(imdb_df.shape)
#print(imdb_df.head())
yelp_df = pd.read_csv(InputFilePath_yelp, sep="\t", header=None, names=["text", "Rating"])
print(yelp_df.shape)
#print(yelp_df.head())

(1000, 2)
(748, 2)
(1000, 2)
```

The datasets were merged into one dataframe

```
#Merging data files - Amazon, imdb, yelp
Sentiment_df= pd.concat((amazon_df,imdb_df,yelp_df),ignore_index=True)
```

- Data exploration

Initial data exploration was undertaken

```
#Data Exploration
Sentiment_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2748 entries, 0 to 2747
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   text    2748 non-null   object
 1   Rating  2748 non-null   int64
dtypes: int64(1), object(1)
memory usage: 43.1+ KB
```

```
Sentiment_df.shape
```

```
|: (2748, 2)
```

```
pd.set_option("display.max_colwidth", None)
Sentiment_df.head()
```

```
|:

```

	text	Rating
0	So there is no way for me to plug it in here in the US unless I go by a converter.	0
1	Good case, Excellent value.	1
2	Great for the jawbone.	1
3	Tied to charger for conversations lasting more than 45 minutes.MAJOR PROBLEMS!!	0
4	The mic is great.	1

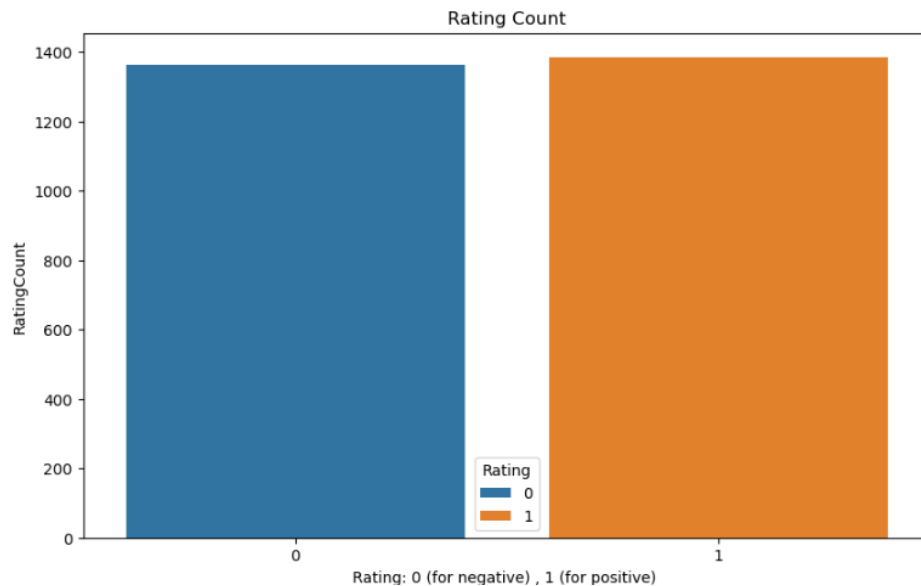
```
#Viewing the sentiments
Sentiment_df['Rating'].value_counts()
#Score is either 1 (for positive) or 0 (for negative)
```

```
|: Rating
1    1386
0    1362
Name: count, dtype: int64
```

```

plt.figure(figsize=(10, 6))
sns.countplot(data=Sentiment_df, x='Rating', hue='Rating')
plt.title('Rating Count')
plt.xlabel('Rating: 0 (for negative) , 1 (for positive) ')
plt.ylabel('RatingCount')
plt.show()

```



Checking for unusual characters, such as emojis and non-common characters

```

#source of regex : https://pythonguides.com/check-if-a-string-is-an-emoji-in-python/

#Checking for unusual characters (e.g., emojis, non-English characters)
#Removing nulls from the data frame and
texts = Sentiment_df["text"].dropna().astype(str).tolist()

```

```

# Regex for emojis
emoji_pattern = re.compile("[
    u\"\\U0001F600-\\U0001F64F\" # identify emojis/emoticons
    u\"\\U0001F300-\\U0001F5FF\" # symbols & pictographs
    u\"\\U0001F680-\\U0001F6FF\" # transport & map symbols
    u\"\\U0001F700-\\U0001F77F\" # alchemical symbols
    u\"\\U0001F780-\\U0001F7FF\" # Geometric Shapes Extended
    u\"\\U0001F800-\\U0001F8FF\" # Supplemental Arrows-C
    u\"\\U0001F900-\\U0001F9FF\" # Supplemental Symbols and Pictographs
    u\"\\U0001FA00-\\U0001FA6F\" # Chess Symbols
    u\"\\U0001FA70-\\U0001FAFF\" # Symbols and Pictographs Extended-A
    u\"\\U0001F1E0-\\U0001F1FF\" # flags
"]+", flags=re.UNICODE)
# Count emojis
emoji_count = sum(len(emoji_pattern.findall(t)) for t in texts)

# Count non-ASCII characters (anything beyond standard English letters & punctuation)
non_ascii_chars = Counter(ch for t in texts for ch in t if ord(ch) > 127)

print("Total emoji occurrences:", emoji_count)
print("Top 15 unusual characters:", non_ascii_chars.most_common(20))

```

Total emoji occurrences: 0

Top 15 unusual characters: [('é', 7), ('\x96', 5), ('\x85', 2), ('â', 1), ('\x97', 1), ('ê', 1)]

Checking for nulls and missing data

```
#Checking for Nulls and missing data

print(Sentiment_df[Sentiment_df[Sentiment_df.columns[Sentiment_df.isna().any()]].columns]

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2748 entries, 0 to 2747
Empty DataFrame
None
```

Checking the vocabulary size

```
#vocabulary size
#Getting the total number of words and characters
Sentiment_df['total Words']=[len(x.split())for x in Sentiment_df['text'].tolist()]
#Sentiment_df['total chars']=Sentiment_df.apply(len)
Sentiment_df
```

	text	Rating	total Words
0	So there is no way for me to plug it in here in the US unless I go by a converter.	0	21
1	Good case, Excellent value.	1	4
2	Great for the jawbone.	1	4
3	Tied to charger for conversations lasting more than 45 minutes.MAJOR PROBLEMS!!	0	11
4	The mic is great.	1	4
...	...	...	...
2743	I think food should have flavor and texture and both were lacking.	0	12
2744	Appetite instantly gone.	0	3
2745	Overall I was not impressed and would not go back.	0	10
2746	The whole experience was underwhelming, and I think we'll just go to Ninja Sushi next time.	0	16
2747	Then, as if I hadn't wasted enough of my life there, they poured salt in the wound by drawing out the time it took to bring the check.	0	28

2748 rows × 3 columns

```
#Length review on the original dataset
text_length = Sentiment_df.text.str.len()
type(text_length)
mean_length = Sentiment_df.text.str.len().mean()
print(f"The mean text Length is: {mean_length} ")
max_length = Sentiment_df.text.str.len().max()
print(f"The Maximum text Length is: {max_length} ")
Min_length = Sentiment_df.text.str.len().min()
print(f"The minimum text Length is: {Min_length} ")
```

The mean text Length is: 71.52838427947599
The Maximum text Length is: 7944
The minimum text Length is: 7

- Data cleaning

In undertaking the data cleaning, the following processes were done

Dropping the missing and the null values

```
#Dropping missing values
Sentiment_df=Sentiment_df.dropna()
```

Removing the URLs and email addresses

```
#Removing URL's and email addresses
Sentiment_df["text"] = Sentiment_df["text"].replace(r"https?://\S+|www\.\S+|\S+@\S+", " ", regex=True)
pd.set_option("display.max_colwidth", None)
Sentiment_df.head()
```

	text	Rating	total Words
0	So there is no way for me to plug it in here in the US unless I go by a converter.	0	21
1	Good case, Excellent value.	1	4
2	Great for the jawbone.	1	4
3	Tied to charger for conversations lasting more than 45 minutes.MAJOR PROBLEMS!!	0	11
4	The mic is great.	1	4

Converting the text to lowercase

```
#Converting to Lower case
Sentiment_df["text"] = Sentiment_df["text"].str.lower()
pd.set_option("display.max_colwidth", None)
Sentiment_df.head()
```

	text	Rating	total Words
0	so there is no way for me to plug it in here in the us unless i go by a converter.	0	21
1	good case, excellent value.	1	4
2	great for the jawbone.	1	4
3	tied to charger for conversations lasting more than 45 minutes.major problems!!	0	11
4	the mic is great.	1	4

Removing punctuation

```
#Remove Punctuations.. Leave letters, numbers and spaces
Sentiment_df["text"] = Sentiment_df["text"].replace(r"[^a-z0-9\s]", " ", regex=True)
pd.set_option("display.max_colwidth", None)
Sentiment_df.head()
```

	text	Rating	total Words
0	so there is no way for me to plug it in here in the us unless i go by a converter	0	21
1	good case excellent value	1	4
2	great for the jawbone	1	4
3	tied to charger for conversations lasting more than 45 minutes major problems	0	11
4	the mic is great	1	4

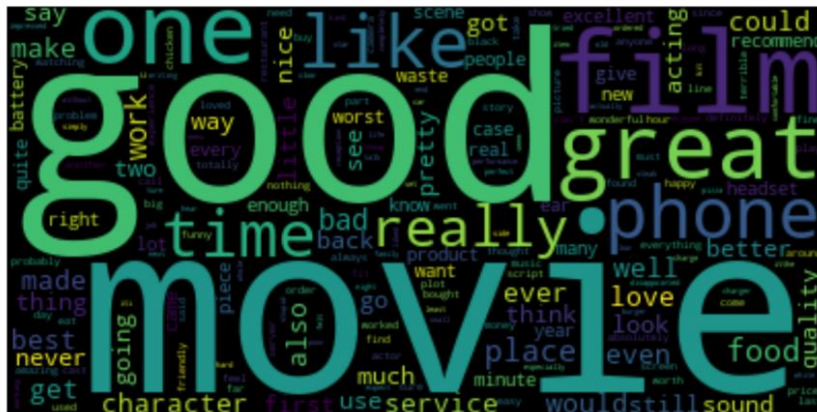
Removing leading and trailing spaces

```
#Removing extra spaces, trailing and leading spaces
Sentiment_df["text"] = Sentiment_df["text"].str.replace(r"\s+", " ", regex=True).str.strip
pd.set_option("display.max_colwidth", None)
Sentiment_df.head()
```

	text	Rating	total Words
0	so there is no way for me to plug it in here in the us unless i go by a converter	0	21
1	good case excellent value	1	4
2	great for the jawbone	1	4
3	tied to charger for conversations lasting more than 45 minutes major problems	0	11
4	the mic is great	1	4

Reviewing the frequently used words

```
#creating a word cloud of the frequently used words
stop_words = set(stopwords.words("english"))
plt.figure(figsize=(20, 15))
FrequentText = ""
for review in review_df.review:
    wordcloud = WordCloud(stopwords=stop_words).generate(FrequentText)
    plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.show()
```



Removing stop words

```
#Removing stop words
#stop_words = set(stopwords.words("english"))

Sentiment_df["text"] = Sentiment_df["text"].apply(
    lambda x: " ".join([word for word in x.split() if word not in stop_words]))

pd.set_option("display.max_colwidth", None)
pd.set_option('display.max_columns', None)
print(Sentiment_df.head())
```

	text	Rating	\
0	way plug us unless go converter	0	
1	good case excellent value	1	
2	great jawbone	1	
3	tied charger conversations lasting 45 minutes major problems	0	
4	mic great	1	

	total Words
0	21
1	4
2	4
3	11
4	4

Finding the vocabulary size

```
#Finding the vocabulary
Sentiment_reviews = Sentiment_df['text'].tolist()
vocabulary = set()
max_review_len = 0

for review in Sentiment_reviews:
    words = review.split()
    max_review_len = max(max_review_len, len(words))
    vocabulary.update(words)
vocab_size = len(vocabulary)

print("Vocabulary Size: ", vocab_size)
print("Longest Review Length: ", max_review_len)
```

Vocabulary Size: 5113
Longest Review Length: 785

Finding the embedding length

```
#Finding the word embedding length

embedding_size = int(np.log2(vocab_size) * 10)
print("Embedding Size: ", embedding_size)
```

Embedding Size: 123

Word tokenization and lemmatization

D213: AdvancedDataAnalytics_Task2_SentimentalAnalysisUsingNeuralNetworks
BeatriceM_WGU_MSDA15

```
# Word Tokenization
#Word Tokenization is the process of splitting text into smaller pieces or tokens.This enables NLP to be able to analyse
# Word Tokenization of the data

Sentiment_df["text_tokenized"] = Sentiment_df["text"].apply(word_tokenize)

#Stemming and Lemmatizing(Matching words)
#Stemming is a text pre-processing technique in NLP that reduces words to their root formby removing prefixes and suffix
#Lemmatization uses linguistics dictionary, rules amd part of speech tags to return actual valid word
#Steming is faster , rule based but it cab produce weird non-words
#Lemati#zation is slower but gives better words for readability and analysis

#Lemmatization of the data

lemmatizer = WordNetLemmatizer()
Sentiment_df["text_lemmatized"] = Sentiment_df["text_tokenized"].apply(
    lambda x: " ".join([lemmatizer.lemmatize(word) for word in x]))

pd.set_option("display.max_colwidth", None)
Sentiment_df.head()
```

	text	Rating	total Words	text_tokenized	text_lemmatized
0	way plug us unless go converter	0	21	[way, plug, us, unless, go, converter]	way plug u unless go conv
1	good case excellent value	1	4	[good, case, excellent, value]	good case excellent v
2	great jawbone	1	4	[great, jawbone]	great jawb
3	tied charger conversations lasting 45 minutes major problems	0	11	[tied, charger, conversations, lasting, 45, minutes, major, problems]	tied charger conversation lasting 45 min major probl
4	mic great	1	4	[mic, great]	mic g

Tokenization

```
#Tokenize the text into individual words
tokenizer =Tokenizer()
tokenizer.fit_on_texts(Sentiment_df["text_lemmatized"])
Sentiment_df["text_sequence"] = tokenizer.texts_to_sequences(Sentiment_df["text_lemmatized"])
Sentiment_df.head()
```

	text	Rating	total Words	text_tokenized	text_lemmatized	text_sequenc
0	way plug us unless go converter	0	21	[way, plug, us, unless, go, converter]	way plug u unless go converter	[44, 252, 103, 476, 2104, :
1	good case excellent value	1	4	[good, case, excellent, value]	good case excellent value	[1, 60, 34, 31:
2	great jawbone	1	4	[great, jawbone]	great jawbone	[3, 96:
3	tied charger conversations lasting 45 minutes major problems	0	11	[tied, charger, conversations, lasting, 45, minutes, major, problems]	tied charger conversation lasting 45 minute major problem	[2044, 126, 632, 964, 132161, 1327, 7:
4	mic great	1	4	[mic, great]	mic great	[759, :

Padding process

```
#Padding process
#Padding is the process of making all sequences the same length
# Set max_len to the 95th percentile
Sentiment_df["sequence_length"] = Sentiment_df["text_sequence"].apply(len)
# Obtaining the quantiles length
quantiles = Sentiment_df["sequence_length"].quantile([0.0, 0.1, 0.25, 0.5, 0.75, 0.9, 0.95, 0.99, 1.0])
print("Sequence Length Quantiles:\n", quantiles)
#Defining the 95th quantile to be used in the padding process
max_len = int(Sentiment_df["sequence_length"].quantile(0.99))
print("95th percentile max_len:", max_len)

Sentiment_padding = pad_sequences(Sentiment_df["text_sequence"], maxlen=max_len, padding='post')
Sentiment_df["text_padded"] = list(Sentiment_padding)

print(Sentiment_df.head(1))

Sequence Length Quantiles:
0.00    0.00
0.10    2.00
0.25    3.00
0.50    5.00
0.75    8.00
0.90   11.30
0.95   14.00
0.99   21.53
1.00  793.00
Name: sequence_length, dtype: float64
95th percentile max_len: 21
      text  Rating  total Words  \
0  way plug us unless go converter      0      21

      text_tokenized      text_lemmatized  \
0  [way, plug, us, unless, go, converter]  way plug u unless go converter

      text_sequence  sequence_length  \
0  [44, 252, 103, 476, 25, 2043]      6

      text_padded
0  [44, 252, 103, 476, 25, 2043, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

print(Sentiment_padding.shape)

(2718, 21)
```

- Splitting the data into a training and a testing set

To ensure reproducibility, the seed was set to 42. The data was split into 80% training and 20% testing sets, whereas in the model fitting, the data is split into 30% as a validation set. This train test split is within the industry standards, with the effective breakdown being 56%training, 24% validation, and 20% testing

```
#Setting the seed
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)
os.environ['TF_DETERMINISTIC_OPS'] = '1'
os.environ['PYTHONHASHSEED'] = '42'

#Split data into training and testing

X = np.array(Sentiment_df["text_padded"].to_list())
y = Sentiment_df["Rating"].values
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)


#Compiling the Model/Training
total_epochs = 20
patience = max(1, int(total_epochs * 0.2)) # 20% of epochs

history=model.fit(X_train, y_train, batch_size=32, epochs=total_epochs,
                  validation_split=0.3, shuffle=False,callbacks=[EarlyStopping(patience=patience , restore_best_weights=True)],
                  verbose=True)
for i in range(len(history.history['loss'])):
    print(f"Epoch {i+1}:",
          f"accuracy={round(history.history['accuracy'][i], 4)}",
          f"loss={round(history.history['loss'][i], 2)}",
          f"val_accuracy={round(history.history['val_accuracy'][i], 4)}",
          f"val_loss={round(history.history['val_loss'][i], 4)}")
```

B6:PREPARED DATA SET

Cleaned dataset



Sentiment_cleaned_
data.txt

Train and Test dataset



X_Test_data.txt



Y_Train_data.txt



y_Test_data.txt



X_Train_data.txt

C1: MODEL SUMMARY

The output of the model summary of the function from TensorFlow.

D213: AdvancedDataAnalytics_Task2_SentimentalAnalysisUsingNeuralNetworks
BeatriceM_WGU_MSDA18

```
I #Building the Neural network Model / ensuring the reproducibility of the results of the model via the seed of 42
word_index_size = len(tokenizer.word_index) + 1
max_len = max_len
print(f"max_len:{max_len}")
model = Sequential()
model.add(Embedding(input_dim=word_index_size, output_dim=embedding_size)) #, input_length=max_len))
model.add(GlobalAveragePooling1D())
model.add(Dropout(0.5))
model.add(Dense(64, activation="relu",kernel_initializer=tf.keras.initializers.GlorotUniform(seed=42)))
model.add(Dropout(0.5))
model.add(Dense(32, activation="relu",kernel_initializer=tf.keras.initializers.GlorotUniform(seed=42)))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid',kernel_initializer=tf.keras.initializers.GlorotUniform(seed=42)))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.build(input_shape=(None, max_len))
model.summary()
```

max_len:21

Model: "sequential_3"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 21, 123)	574,041
global_average_pooling1d_3 (GlobalAveragePooling1D)	(None, 123)	0
dropout_9 (Dropout)	(None, 123)	0
dense_9 (Dense)	(None, 64)	7,936
dropout_10 (Dropout)	(None, 64)	0
dense_10 (Dense)	(None, 32)	2,080
dropout_11 (Dropout)	(None, 32)	0
dense_11 (Dense)	(None, 1)	33

Total params: 584,090 (2.23 MB)

Trainable params: 584,090 (2.23 MB)

Non-trainable params: 0 (0.00 B)

```
#Compiling the Model/Training
total_epochs = 20
patience = max(1, int(total_epochs * 0.2)) # 20% of epochs

history=model.fit(X_train, y_train, batch_size=32, epochs=total_epochs,
                  validation_split=0.3, shuffle=False,callbacks=[EarlyStopping(patience=patience , restore_best_weights=True)],
                  verbose=True)
for i in range(len(history.history['loss'])):
    print(f"Epoch {i+1}:",
          f"accuracy={round(history.history['accuracy'][i], 4)}",
          f"loss={round(history.history['loss'][i], 2)}",
          f"val_accuracy={round(history.history['val_accuracy'][i], 4)}",
          f"val_loss={round(history.history['val_loss'][i], 4)}")
```

```
Epoch 1/20
49/49 ----- 2s 14ms/step - accuracy: 0.4915 - loss: 0.6950 - val_accuracy: 0.5258 - val_loss: 0.6924
Epoch 2/20
49/49 ----- 0s 8ms/step - accuracy: 0.5241 - loss: 0.6913 - val_accuracy: 0.5258 - val_loss: 0.6901
Epoch 3/20
49/49 ----- 1s 7ms/step - accuracy: 0.5098 - loss: 0.6931 - val_accuracy: 0.5303 - val_loss: 0.6890
Epoch 4/20
49/49 ----- 0s 7ms/step - accuracy: 0.5696 - loss: 0.6838 - val_accuracy: 0.5348 - val_loss: 0.6805
Epoch 5/20
49/49 ----- 0s 5ms/step - accuracy: 0.6086 - loss: 0.6730 - val_accuracy: 0.6212 - val_loss: 0.6600
Epoch 6/20
49/49 ----- 0s 7ms/step - accuracy: 0.6912 - loss: 0.6249 - val_accuracy: 0.6561 - val_loss: 0.6134
Epoch 7/20
49/49 ----- 0s 8ms/step - accuracy: 0.7939 - loss: 0.5178 - val_accuracy: 0.7348 - val_loss: 0.5246
Epoch 8/20
49/49 ----- 0s 5ms/step - accuracy: 0.8570 - loss: 0.3825 - val_accuracy: 0.7894 - val_loss: 0.4758
Epoch 9/20
49/49 ----- 1s 10ms/step - accuracy: 0.8979 - loss: 0.2978 - val_accuracy: 0.7667 - val_loss: 0.4876
Epoch 10/20
49/49 ----- 0s 9ms/step - accuracy: 0.9239 - loss: 0.2259 - val_accuracy: 0.7545 - val_loss: 0.5226
Epoch 11/20
49/49 ----- 0s 8ms/step - accuracy: 0.9187 - loss: 0.2085 - val_accuracy: 0.7879 - val_loss: 0.5170
Epoch 12/20
49/49 ----- 1s 10ms/step - accuracy: 0.9298 - loss: 0.1996 - val_accuracy: 0.7288 - val_loss: 0.6675
Epoch 1: accuracy=0.4915 loss=0.7 val_accuracy=0.5258 val_loss=0.6924
Epoch 2: accuracy=0.5241 loss=0.69 val_accuracy=0.5258 val_loss=0.6901
Epoch 3: accuracy=0.5098 loss=0.69 val_accuracy=0.5303 val_loss=0.689
Epoch 4: accuracy=0.5696 loss=0.68 val_accuracy=0.5348 val_loss=0.6805
Epoch 5: accuracy=0.6086 loss=0.67 val_accuracy=0.6212 val_loss=0.66
Epoch 6: accuracy=0.6912 loss=0.62 val_accuracy=0.6561 val_loss=0.6134
Epoch 7: accuracy=0.7939 loss=0.52 val_accuracy=0.7348 val_loss=0.5246
Epoch 8: accuracy=0.857 loss=0.38 val_accuracy=0.7894 val_loss=0.4758
Epoch 9: accuracy=0.8979 loss=0.3 val_accuracy=0.7667 val_loss=0.4876
Epoch 10: accuracy=0.9239 loss=0.23 val_accuracy=0.7545 val_loss=0.5226
Epoch 11: accuracy=0.9187 loss=0.21 val_accuracy=0.7879 val_loss=0.517
Epoch 12: accuracy=0.9298 loss=0.2 val_accuracy=0.7288 val_loss=0.6675
```

C2:NETWORK ARCHITECTURE

I designed a simple neural network model that utilizes average pooling with multiple dropout layers to prevent overfitting. The architecture overview of the model was created using the following layers: input layer/ embedding layer, hidden layers that consisted of global average pooling 1D, dropout layer, dense layers, and output layer. The layers are covered as follows

- Input Layer. The input layer receives raw data. The input layer had a sequence length of 21, which represents the 99th percentile sequence.

- **Embedding Layer.** The embedding layer converts each token into a dense vector of numbers. For the purpose of this model, each token is represented by a 123-dimensional vector. For the vocabulary size, the model recognized 4,667 unique tokens/words. The output shape for each batch was (batch_size, 21, 123). The embedding layer has 123 dimensions. The vocabulary size is 4667 words. The number of trainable parameters in the embedding layer is 574041.
- **Global Average Pooling 1D** was utilized to perform dimensionality reduction and prevent overfitting. The Global Average Pooling 1D takes the average of each feature across all time steps. It compresses the sequence into a global representation, removing positional information but retaining overall meaning. In this model, the Global Average Pooling 1D averages the 21 embeddings into a single vector of size 123. This reduces sequence information to a global representation.
- **Dropout Layer** was utilized to randomly disable a fraction of the features during training and set a fraction of features to zero during training, hence helping in preventing overfitting.
- **Dense Layer (64 units, ReLU activation).** This Layer Learns abstract patterns from the pooled embedding and utilizes the activation ReLU (Rectified Linear Unit), which introduces non-linearity, helping the model learn complex relationships. This layer had 7936 trainable parameters
- **Dropout Layer** was utilized to randomly disable a fraction of the features during training and set a fraction of features to zero during training, hence helping in preventing overfitting.

- Dense Layer (32 units, ReLU activation) was utilized to transform the representation into a more compact feature space and sets a fraction of features to zero during training, hence helping in preventing overfitting. This dense layer had 2080 trainable parameters.
- Dropout Layer was utilized to randomly disable a fraction of the features during training and set a fraction of features to zero during training, hence helping in preventing overfitting.
- Dense Output Layer (1 unit) was utilized to produce the final prediction. For binary classification, the sigmoid activation function was used, while linear activation was used for regression. The final dense layer had 33 trainable parameters

Parameters refer to the weights and biases that the model learns during training. In this analysis, the total parameters, which represent the total number of weights and biases, were 584090; the trainable parameters, which represent the parameters that the model actively learns and updates, were 584090, which means that all the parameters were trainable and there were no non-trainable parameters.

max_len:21

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 21, 123)	574,041
global_average_pooling1d (GlobalAveragePooling1D)	(None, 123)	0
dropout (Dropout)	(None, 123)	0
dense (Dense)	(None, 64)	7,936
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_2 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 1)	33

Total params: 584,090 (2.23 MB)

Trainable params: 584,090 (2.23 MB)

Non-trainable params: 0 (0.00 B)

C3:HYPERPARAMETERS

In a machine learning Model, hyperparameters are parameters that control the learning process.

- activation functions

An activation function is a mathematical operation used in neural networks to introduce nonlinearity into a model. The activation function enables a neural network model to learn and represent complex relationships, helping the model determine which neuron to activate based on the input signals. The model in this case utilized ReLU and sigmoid functions.

ReLU was chosen for the dense layers, which is used to introduce nonlinearity in the model, allowing it to learn sparse and discriminative features. More so, it is efficient in utilizing computational resources and reduces vanishing gradient problems.

Sigmoid was chosen for the output layer since it is a binary classification task that maps the outputs in the negative and positive ratings, aligning it to the binary cross-entropy loss.

```
model.add(Dropout(0.5))  
model.add(Dense(64, activation="relu", kernel_initializer=tf.keras.initializers.GlorotUniform(seed=42)))  
model.add(Dropout(0.5))  
model.add(Dense(32, activation="relu", kernel_initializer=tf.keras.initializers.GlorotUniform(seed=42)))  
model.add(Dropout(0.5))  
model.add(Dense(1, activation='sigmoid', kernel_initializer=tf.keras.initializers.GlorotUniform(seed=42)))
```

- number of nodes per layer

In neural networks, the number of nodes per layer refers to the number of computational units, or neurons, in each layer of the network. A node or neuron is a unit that receives the input, applies a mathematical transformation, and passes the results to the next layer. Each node allows the neural network to learn the patterns and make predictions.

The neural network model that I created had the following layers

- The input layer had 21 nodes, and the embedding layer had 123 nodes
- Global Average pooling1D had 123 nodes

- The Hidden layer had Dense layer 1 with 64 nodes, which allowed the model to capture many features from the embedding model, and Dense layer 2 had 32 nodes, which reduced dimensionality and preserved the relevant patterns.
- The output layer had 1 node

max_len:21

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 21, 123)	574,041
global_average_pooling1d (GlobalAveragePooling1D)	(None, 123)	0
dropout (Dropout)	(None, 123)	0
dense (Dense)	(None, 64)	7,936
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dropout_2 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 1)	33

Total params: 584,090 (2.23 MB)

Trainable params: 584,090 (2.23 MB)

Non-trainable params: 0 (0.00 B)

- loss function

A loss function in a machine learning model is a mathematical formula used to measure how well a model's prediction matches the actual results. The loss function utilized in this model is binary cross-entropy for binary classification. Binary cross-entropy is also called log loss. The data provided consisted of either positive or negative ratings, and therefore, the binary cross-entropy is suitable for this model. Moreover, it works well with sigmoid output.

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

- optimizer

An optimizer is an algorithm used in machine learning to adjust the models' parameters, such as weights and biases, so as to minimize the loss function. In this model, the Adam optimizer was utilized. The Adam optimizer was chosen because it dynamically adapts the learning rate for each parameter, combining the benefits of AdaGrad and RMSProp, making it robust for sparse features, which are common in text embeddings, and thereby accelerating convergence. The Adam optimizer is the most popular and effective optimizer in deep learning.

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

- stopping criteria

Stopping criteria are the rules and conditions that determine when to terminate the training of a machine learning model, helping to strike a balance between overfitting and underfitting. In this model, the Early stopping criteria were utilized. The early stopping criteria set the rules for evaluating the model, which stops when the model's performance on the validation set stops improving. It monitors validation loss or accuracy and will stop if it notices no improvement after a specified number of epochs or a set patience. An epoch is one complete pass through the entire dataset. In this analysis, the total epoch was set to 20, and early stopping with patience was set to 4 epochs (20% of the total epochs). It also incorporated the `restore_best_weights` parameter, setting it to true, ensuring that the final model corresponds to the epoch with the best validation performance. Below are the stopping criteria for this model

```

#Compiling the Model/Training
total_epochs = 20
patience = max(1, int(total_epochs * 0.2)) # 20% of epochs

history=model.fit(X_train, y_train, batch_size=32, epochs=total_epochs,
                  validation_split=0.3, shuffle=False,callbacks=[EarlyStopping(patience=patience , restore_best_weights=True)],
                  verbose=True)
for i in range(len(history.history['loss'])):
    print(f"Epoch {i+1}:",
          f"accuracy={round(history.history['accuracy'][i], 4)}",
          f"loss={round(history.history['loss'][i], 2)}",
          f"val_accuracy={round(history.history['val_accuracy'][i], 4)}",
          f"val_loss={round(history.history['val_loss'][i], 4)}")

```

Epoch 1/20
 49/49 ————— 2s 14ms/step - accuracy: 0.4915 - loss: 0.6950 - val_accuracy: 0.5258 - val_loss: 0.6924
 Epoch 2/20
 49/49 ————— 0s 8ms/step - accuracy: 0.5241 - loss: 0.6913 - val_accuracy: 0.5258 - val_loss: 0.6901
 Epoch 3/20
 49/49 ————— 1s 7ms/step - accuracy: 0.5098 - loss: 0.6931 - val_accuracy: 0.5303 - val_loss: 0.6890
 Epoch 4/20
 49/49 ————— 0s 7ms/step - accuracy: 0.5696 - loss: 0.6838 - val_accuracy: 0.5348 - val_loss: 0.6805
 Epoch 5/20
 49/49 ————— 0s 5ms/step - accuracy: 0.6086 - loss: 0.6730 - val_accuracy: 0.6212 - val_loss: 0.6600
 Epoch 6/20
 49/49 ————— 0s 7ms/step - accuracy: 0.6912 - loss: 0.6249 - val_accuracy: 0.6561 - val_loss: 0.6134
 Epoch 7/20
 49/49 ————— 0s 8ms/step - accuracy: 0.7939 - loss: 0.5178 - val_accuracy: 0.7348 - val_loss: 0.5246
 Epoch 8/20
 49/49 ————— 0s 5ms/step - accuracy: 0.8570 - loss: 0.3825 - val_accuracy: 0.7894 - val_loss: 0.4758
 Epoch 9/20
 49/49 ————— 1s 10ms/step - accuracy: 0.8979 - loss: 0.2978 - val_accuracy: 0.7667 - val_loss: 0.4876
 Epoch 10/20
 49/49 ————— 0s 9ms/step - accuracy: 0.9239 - loss: 0.2259 - val_accuracy: 0.7545 - val_loss: 0.5226
 Epoch 11/20
 49/49 ————— 0s 8ms/step - accuracy: 0.9187 - loss: 0.2085 - val_accuracy: 0.7879 - val_loss: 0.5170
 Epoch 12/20
 49/49 ————— 1s 10ms/step - accuracy: 0.9298 - loss: 0.1996 - val_accuracy: 0.7288 - val_loss: 0.6675
 Epoch 1: accuracy=0.4915 loss=0.7 val_accuracy=0.5258 val_loss=0.6924
 Epoch 2: accuracy=0.5241 loss=0.69 val_accuracy=0.5258 val_loss=0.6901
 Epoch 3: accuracy=0.5098 loss=0.69 val_accuracy=0.5303 val_loss=0.689
 Epoch 4: accuracy=0.5696 loss=0.68 val_accuracy=0.5348 val_loss=0.6805
 Epoch 5: accuracy=0.6086 loss=0.67 val_accuracy=0.6212 val_loss=0.66
 Epoch 6: accuracy=0.6912 loss=0.62 val_accuracy=0.6561 val_loss=0.6134
 Epoch 7: accuracy=0.7939 loss=0.52 val_accuracy=0.7348 val_loss=0.5246
 Epoch 8: accuracy=0.857 loss=0.38 val_accuracy=0.7894 val_loss=0.4758
 Epoch 9: accuracy=0.8979 loss=0.3 val_accuracy=0.7667 val_loss=0.4876
 Epoch 10: accuracy=0.9239 loss=0.23 val_accuracy=0.7545 val_loss=0.5226
 Epoch 11: accuracy=0.9187 loss=0.21 val_accuracy=0.7879 val_loss=0.517
 Epoch 12: accuracy=0.9298 loss=0.2 val_accuracy=0.7288 val_loss=0.6675

- evaluation metric

An evaluation metric is a quantitative measure used to assess how a machine learning model performs on a given task, helping to determine the quality of the model's predictions. It also guides model selection, tuning, and deployment decisions. The final accuracy for this model is 79.09% which indicates that the model generalizes reasonably well.

```
#Evaluating the performance
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy}")
print(f"Test Loss: {loss}")
```

18/18 ————— 0s 5ms/step - accuracy: 0.7909 - loss: 0.4556
Test Accuracy: 0.7909091114997864
Test Loss: 0.4556426703929901

D1: STOPPING CRITERIA

Stopping criteria are the rules and conditions that determine when to terminate the training of a machine learning model, and they help strike a balance between overfitting and underfitting. In this model, the Early stopping criteria were utilized. The early stopping criteria set the rules for evaluating the model, which stops when the model's performance on the validation set stops improving. It monitors validation loss or accuracy and will stop if it notices no improvement after a few epochs or a specified patience. An epoch is one complete pass through the entire dataset. In this analysis, the total epoch was set to 20, and early stopping with patience was set to 4 epochs (20% of the total epochs). It also incorporated the `restore_best_weights` parameter to true, ensuring that the final model corresponds to the epoch with the best validation performance. The screenshot for the stopping criteria for this model

```
#Compiling the Model/Training
total_epochs = 20
patience = max(1, int(total_epochs * 0.2)) # 20% of epochs

history=model.fit(X_train, y_train, batch_size=32, epochs=total_epochs,
                  validation_split=0.3, shuffle=False,callbacks=[EarlyStopping(patience=patience , restore_best_weights=True)],
                  verbose=True)
for i in range(len(history.history['loss'])):
    print(f"Epoch {i+1}:",
          f"accuracy={round(history.history['accuracy'][i], 4)}",
          f"loss={round(history.history['loss'][i], 2)}",
          f"val_accuracy={round(history.history['val_accuracy'][i], 4)}",
          f"val_loss={round(history.history['val_loss'][i], 4)}")
```

Epoch 1/20
49/49 ————— 2s 14ms/step - accuracy: 0.4915 - loss: 0.6950 - val_accuracy: 0.5258 - val_loss: 0.6924
Epoch 2/20
49/49 ————— 0s 8ms/step - accuracy: 0.5241 - loss: 0.6913 - val_accuracy: 0.5258 - val_loss: 0.6901
Epoch 3/20
49/49 ————— 1s 7ms/step - accuracy: 0.5098 - loss: 0.6931 - val_accuracy: 0.5303 - val_loss: 0.6890
Epoch 4/20
49/49 ————— 0s 7ms/step - accuracy: 0.5696 - loss: 0.6838 - val_accuracy: 0.5348 - val_loss: 0.6805
Epoch 5/20
49/49 ————— 0s 5ms/step - accuracy: 0.6086 - loss: 0.6730 - val_accuracy: 0.6212 - val_loss: 0.6600
Epoch 6/20
49/49 ————— 0s 7ms/step - accuracy: 0.6912 - loss: 0.6249 - val_accuracy: 0.6561 - val_loss: 0.6134
Epoch 7/20
49/49 ————— 0s 8ms/step - accuracy: 0.7939 - loss: 0.5178 - val_accuracy: 0.7348 - val_loss: 0.5246
Epoch 8/20
49/49 ————— 0s 5ms/step - accuracy: 0.8570 - loss: 0.3825 - val_accuracy: 0.7894 - val_loss: 0.4758
Epoch 9/20
49/49 ————— 1s 10ms/step - accuracy: 0.8979 - loss: 0.2978 - val_accuracy: 0.7667 - val_loss: 0.4876
Epoch 10/20
49/49 ————— 0s 9ms/step - accuracy: 0.9239 - loss: 0.2259 - val_accuracy: 0.7545 - val_loss: 0.5226
Epoch 11/20
49/49 ————— 0s 8ms/step - accuracy: 0.9187 - loss: 0.2085 - val_accuracy: 0.7879 - val_loss: 0.5170
Epoch 12/20
49/49 ————— 1s 10ms/step - accuracy: 0.9298 - loss: 0.1996 - val_accuracy: 0.7288 - val_loss: 0.6675
Epoch 1: accuracy=0.4915 loss=0.7 val_accuracy=0.5258 val_loss=0.6924
Epoch 2: accuracy=0.5241 loss=0.69 val_accuracy=0.5258 val_loss=0.6901
Epoch 3: accuracy=0.5098 loss=0.69 val_accuracy=0.5303 val_loss=0.689
Epoch 4: accuracy=0.5696 loss=0.68 val_accuracy=0.5348 val_loss=0.6805
Epoch 5: accuracy=0.6086 loss=0.67 val_accuracy=0.6212 val_loss=0.66
Epoch 6: accuracy=0.6912 loss=0.62 val_accuracy=0.6561 val_loss=0.6134
Epoch 7: accuracy=0.7939 loss=0.52 val_accuracy=0.7348 val_loss=0.5246
Epoch 8: accuracy=0.857 loss=0.38 val_accuracy=0.7894 val_loss=0.4758
Epoch 9: accuracy=0.8979 loss=0.3 val_accuracy=0.7667 val_loss=0.4876
Epoch 10: accuracy=0.9239 loss=0.23 val_accuracy=0.7545 val_loss=0.5226
Epoch 11: accuracy=0.9187 loss=0.21 val_accuracy=0.7879 val_loss=0.517
Epoch 12: accuracy=0.9298 loss=0.2 val_accuracy=0.7288 val_loss=0.6675

D2: FITNESS

In assessing the model performance, the following metrics were assessed

- Training performance. In comparing epoch 1 to epoch 8, accuracy improved from 49.15% to 85.7% whereas the loss decreased steadily
- The validation accuracy peaked at 78.94% while the validation loss started to increase after epoch 8, showing signs of over-fitting
- The test performance gave a final accuracy of 79.09% and a loss of 45.56% which shows that the model performs well on unseen data, with the performance being closer to the validation accuracy.

To address overfitting, the following measures were taken

- I added a dropout layer of 0.5 after each layer so as to randomly deactivate 50% of the neurons during the training and prevent feature adaptation
- Early stopping with a patience of 4 restores the best weights, which helps stop training when the loss stops improving.
- I used a validation split of 30% which gives a reliable measure of generalization during training
- Fixing the random seed at 42 ensured reproducibility

The dropout and early stopping mechanisms effectively managed the overfitting that began to appear after epoch 8, allowing the model to strike a balance between performance and generalization.

D3: TRAINING PROCESS

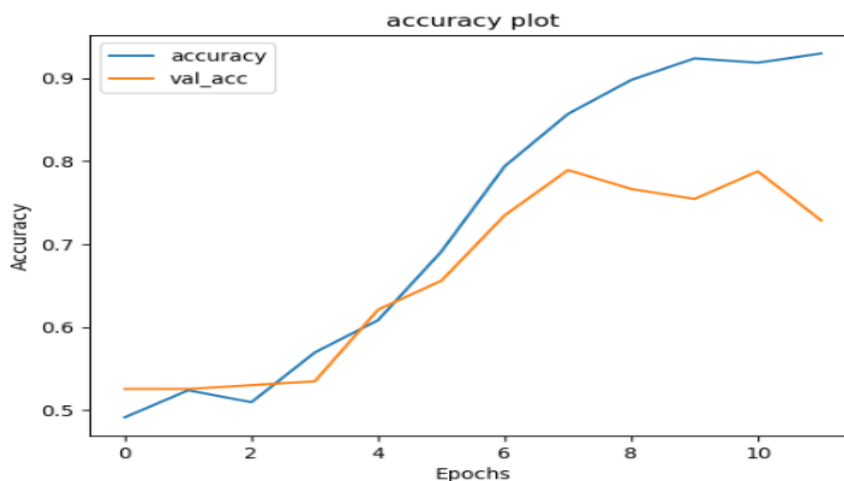
Training performance. In comparing epoch 1 to epoch 8, accuracy improved from 49.15% to 85.7% whereas the loss decreased steadily. At epoch 9, we observe that the validation accuracy begins to decline, while the validation loss starts to increase, indicating overfitting.

```
#Compiling the Model/Training
total_epochs = 20
patience = max(1, int(total_epochs * 0.2)) # 20% of epochs

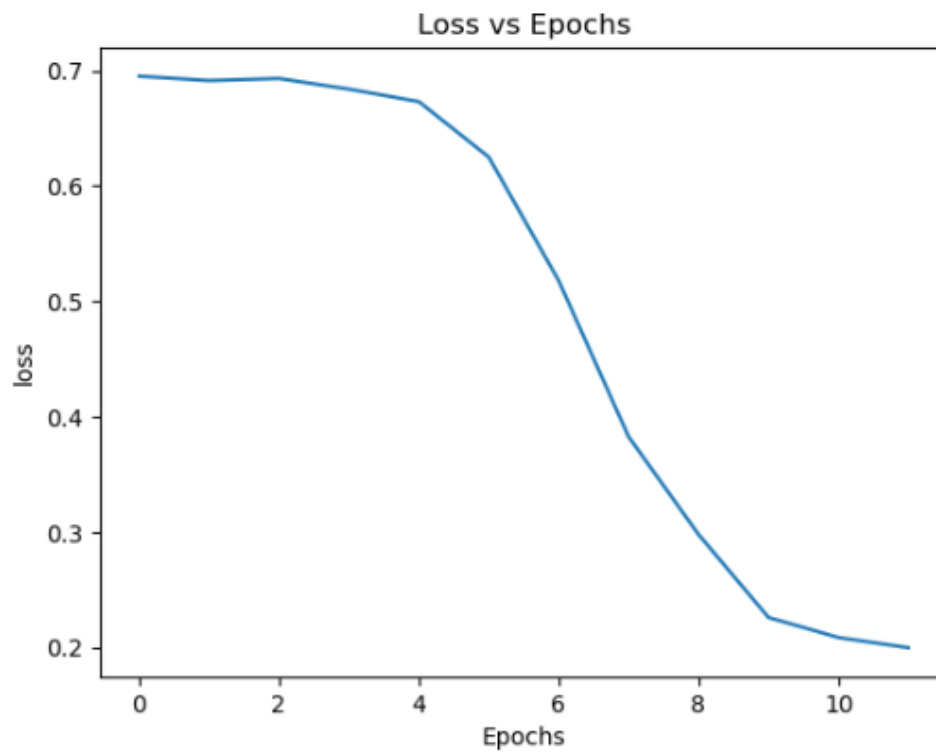
history=model.fit(X_train, y_train, batch_size=32, epochs=total_epochs,
                  validation_split=0.3, shuffle=False,callbacks=[EarlyStopping(patience=patience , restore_best_weights=True)],
                  verbose=True)
for i in range(len(history.history['loss'])):
    print(f"Epoch {i+1}:",
          f"accuracy={round(history.history['accuracy'][i], 4)}",
          f"loss={round(history.history['loss'][i], 2)}",
          f"val_accuracy={round(history.history['val_accuracy'][i], 4)}",
          f"val_loss={round(history.history['val_loss'][i], 4)}")
```

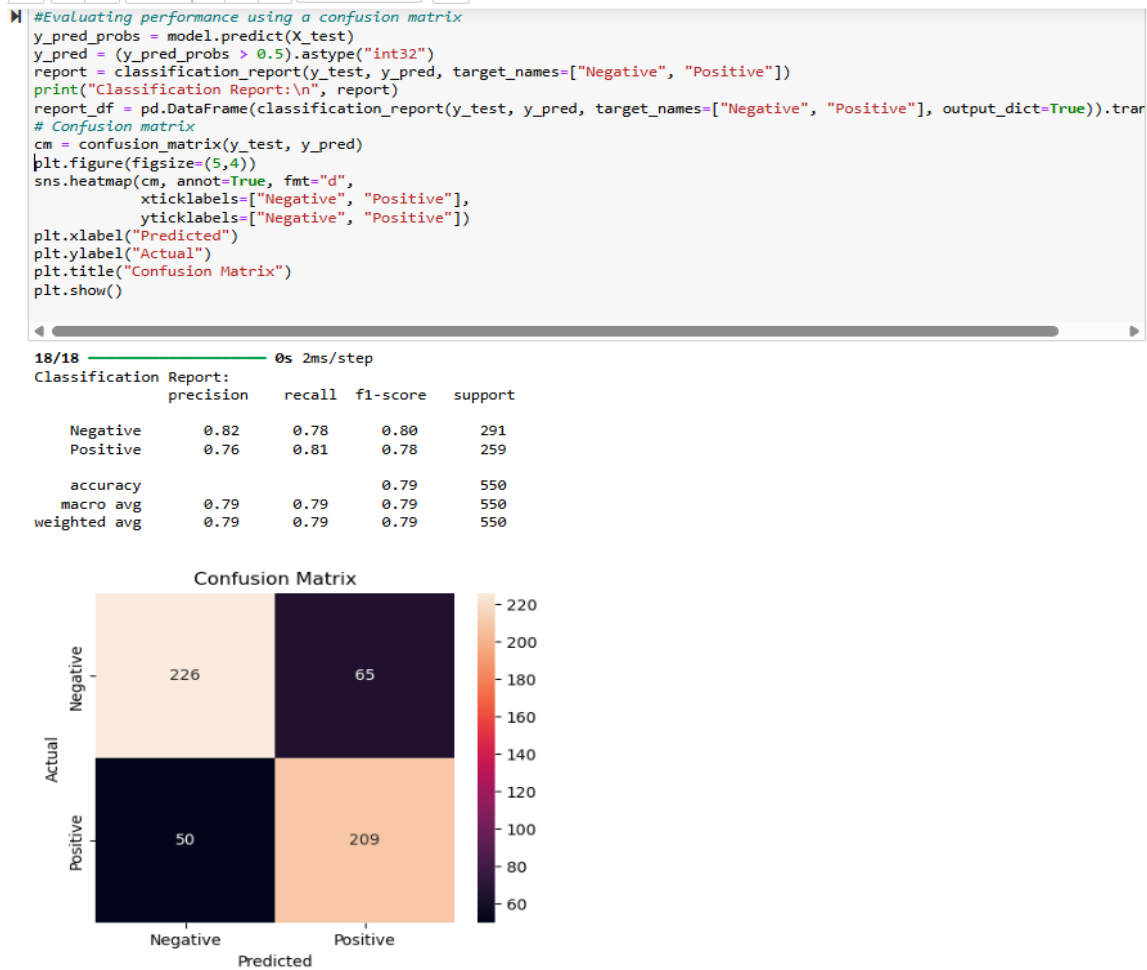
```
Epoch 1/20
49/49 ----- 2s 14ms/step - accuracy: 0.4915 - loss: 0.6950 - val_accuracy: 0.5258 - val_loss: 0.6924
Epoch 2/20
49/49 ----- 0s 8ms/step - accuracy: 0.5241 - loss: 0.6913 - val_accuracy: 0.5258 - val_loss: 0.6901
Epoch 3/20
49/49 ----- 0s 8ms/step - accuracy: 0.5098 - loss: 0.6931 - val_accuracy: 0.5303 - val_loss: 0.6890
Epoch 4/20
49/49 ----- 0s 8ms/step - accuracy: 0.5696 - loss: 0.6838 - val_accuracy: 0.5348 - val_loss: 0.6805
Epoch 5/20
49/49 ----- 0s 8ms/step - accuracy: 0.6086 - loss: 0.6730 - val_accuracy: 0.6212 - val_loss: 0.6600
Epoch 6/20
49/49 ----- 0s 7ms/step - accuracy: 0.6912 - loss: 0.6249 - val_accuracy: 0.6561 - val_loss: 0.6134
Epoch 7/20
49/49 ----- 0s 8ms/step - accuracy: 0.7939 - loss: 0.5178 - val_accuracy: 0.7348 - val_loss: 0.5246
Epoch 8/20
49/49 ----- 0s 8ms/step - accuracy: 0.8570 - loss: 0.3825 - val_accuracy: 0.7894 - val_loss: 0.4758
Epoch 9/20
49/49 ----- 0s 9ms/step - accuracy: 0.8979 - loss: 0.2978 - val_accuracy: 0.7667 - val_loss: 0.4876
Epoch 10/20
49/49 ----- 0s 7ms/step - accuracy: 0.9239 - loss: 0.2259 - val_accuracy: 0.7545 - val_loss: 0.5226
Epoch 11/20
49/49 ----- 0s 9ms/step - accuracy: 0.9187 - loss: 0.2085 - val_accuracy: 0.7879 - val_loss: 0.5170
Epoch 12/20
49/49 ----- 0s 7ms/step - accuracy: 0.9298 - loss: 0.1996 - val_accuracy: 0.7288 - val_loss: 0.6675
Epoch 1: accuracy=0.4915 loss=0.7 val_accuracy=0.5258 val_loss=0.6924
Epoch 2: accuracy=0.5241 loss=0.69 val_accuracy=0.5258 val_loss=0.6901
Epoch 3: accuracy=0.5098 loss=0.69 val_accuracy=0.5303 val_loss=0.689
Epoch 4: accuracy=0.5696 loss=0.68 val_accuracy=0.5348 val_loss=0.6805
Epoch 5: accuracy=0.6086 loss=0.67 val_accuracy=0.6212 val_loss=0.66
Epoch 6: accuracy=0.6912 loss=0.62 val_accuracy=0.6561 val_loss=0.6134
Epoch 7: accuracy=0.7939 loss=0.52 val_accuracy=0.7348 val_loss=0.5246
Epoch 8: accuracy=0.857 loss=0.38 val_accuracy=0.7894 val_loss=0.4758
Epoch 9: accuracy=0.8979 loss=0.3 val_accuracy=0.7667 val_loss=0.4876
Epoch 10: accuracy=0.9239 loss=0.23 val_accuracy=0.7545 val_loss=0.5226
Epoch 11: accuracy=0.9187 loss=0.21 val_accuracy=0.7879 val_loss=0.517
Epoch 12: accuracy=0.9298 loss=0.2 val_accuracy=0.7288 val_loss=0.6675
```

```
#Accuracy plots
plt.plot(history.history['accuracy'], label = 'accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_acc')
plt.legend()
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('accuracy plot')
plt.show()
```



```
#Loss plot  
plt.plot(history.history['loss'])  
plt.xlabel('Epochs')  
plt.ylabel('loss')  
plt.title('Loss vs Epochs')  
plt.show()
```





D4: PREDICTIVE ACCURACY

The validation accuracy peaked at 78.94%, while the validation loss began to increase after epoch 8, indicating signs of overfitting. The test performance yielded a final accuracy of 79.09% and a loss of 45.56%, indicating that the model performs well on unseen data, with its performance being closer to the validation accuracy. When a confusion matrix was plotted, it confirmed that the model accurately classifies 79% of the text reviews.

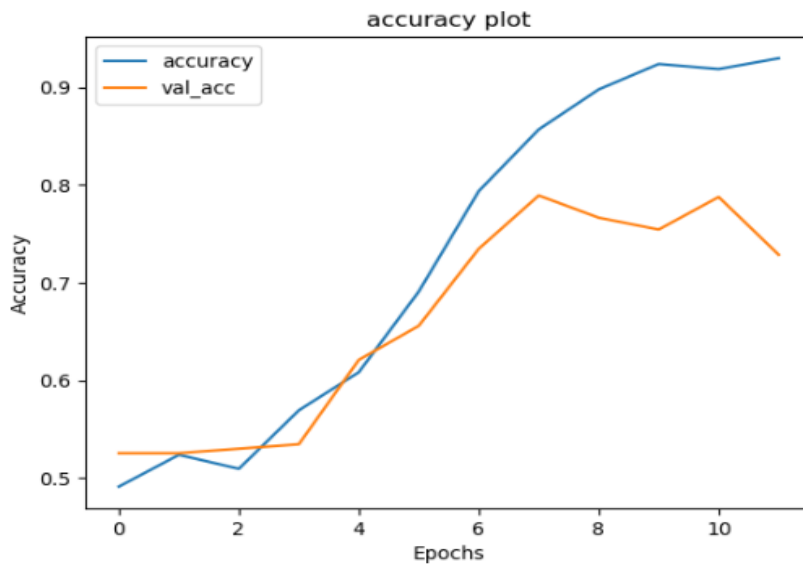
```
#Evaluating the performance
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy}")
print(f"Test Loss: {loss}")
```

18/18 ————— 0s 3ms/step - accuracy: 0.7782 - loss: 0.5151

Test Accuracy: 0.778181791305542

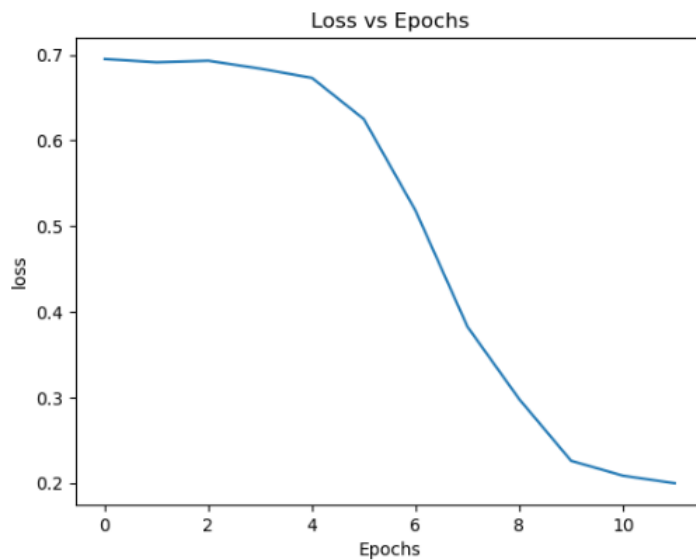
Test Loss: 0.5150659084320068

```
#Accuracy plots
plt.plot(history.history['accuracy'], label = 'accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_acc')
plt.legend()
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('accuracy plot')
plt.show()
```



a line graph of the loss and chosen evaluation metric.

```
#Loss plot
plt.plot(history.history['loss'])
plt.xlabel('Epochs')
plt.ylabel('loss')
plt.title('Loss vs Epochs')
plt.show()
```



E: CODE

Please modify the local file path in the code

```
Local_path = 'C:/Users/beatr/OneDrive/Desktop/WGU/D213/Task2/Data/sentiment labelled  
sentences/'
```



F: FUNCTIONALITY

The model achieved a 79% test accuracy rate in capturing semantic information for sentiment analysis while controlling overfitting. This efficiency was achieved through the neural network architecture, which included embedding layers and 1D Global average pooling, allowing for the efficient capture of overall word meaning. The model also included dense layers (64 and 32) that enabled the model to learn hierarchical feature representations from the text embeddings, and the second layer facilitated compression to prevent over-parametrization. Dropout layers were used to prevent overfitting. Due to the efficiency in the architecture, the model was able to learn after 5 epochs. Early dropping in the model allowed it to avoid overfitting during training. Based on the model's performance, we can utilize it to perform sentiment analysis on text reviews from the combined data of Amazon, IMDb, and Yelp, using a neural network model and NLP techniques. This enables us to understand areas for improvement based on our customers' feedback and enhance customer satisfaction.

G: RECOMMENDATIONS

Based on the performance of the simple neural network model in classifying the sentiment reviews for Amazon, IMDb, and Yelp, I recommend that the model be adopted to carry out analysis of the customers' sentiment reviews with an aim of understanding the areas of

improvement that the customers have indicated in their reviews and enhancing customer satisfaction. However, since the accuracy rate was 79%, the model may require further enhancement to improve the accuracy rate and reduce the overfitting observed at epoch 9.

H: REPORTING

Code in Jupyter Notebook for the Neural Network model developed in the Python is attached



BeatriceMwangi_D2
13 Advanced Data A

PDF document of the executed notebook presentation



BeatriceMwangi_D2
13 Advanced Data A

I: SOURCES FOR THIRD-PARTY CODE

Kumar, Bijay (2025, January 23), How to Check if a String is an Emoji in Python?

<https://pythonguides.com/check-if-a-string-is-an-emoji-in-python/>

J: SOURCES

Virahonda, Sergio (2020, October 9), An easy tutorial about Sentiment Analysis with Deep Learning and Keras

<https://towardsdatascience.com/an-easy-tutorial-about-sentiment-analysis-with-deep-learning-and-keras-2bf52b9cba91/>

Geek for geeks (2025, July 12), What is sentiment Analysis?

<https://www.geeksforgeeks.org/machine-learning/what-is-sentiment-analysis/>

Zakaria, Ahmed (), Tokenization: The Cornerstone for NLP Tasks

<https://mlarchive.com/natural-language-processing/tokenization-the-cornerstone-for-nlp-tasks/>

K: PROFESSIONAL COMMUNICATION